O que o algoritmo faz, quantas operações executa e como cresce?

Sequência da aula

Problema real $\longrightarrow$ **pseudocódigo** $\longrightarrow$ **simulação** $\longrightarrow$ $C(n)$ $\longrightarrow$ **prova assintótica**

Crescimento de funções, contagem de operações e notações assintóticas.

Objetivos de aprendizagem

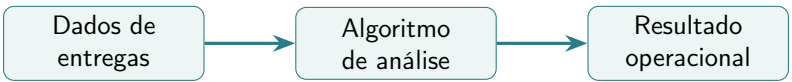
Ao final da aula, espera-se que o estudante consiga:

1. Definir **o tamanho da entrada** em um problema concreto.
2. Identificar uma **operação básica** a ser contabilizada.
3. Simular pequenas instâncias e obter uma **função exata de contagem**.
4. Explicar o crescimento constante, linear, logarítmico, linearítmico, quadrático, cúbico, exponencial e fatorial.
5. Demonstrar limites com O, Ω, Θ e o.

Contexto: sistema de gestão de entregas

Cenário comum aos oito exemplos

Uma plataforma de logística gerencia encomendas, faturas, transportadoras, clientes e rotas de entrega. O mesmo sistema executa tarefas de natureza muito diferente.



Foco

Não compararemos apenas tarefas: analisaremos **como a quantidade de operações varia** quando o volume de dados aumenta.

A unidade de análise: o que é n?

- n pode representar a quantidade de faturas, pedidos ou pontos de entrega.
- Cada algoritmo precisa de uma **definição explícita de** n.
- Dois algoritmos com o mesmo n podem realizar quantidades de trabalho muito diferentes.

Exemplo

Se uma lista contém oito encomendas, então $n = 8$. Se um algoritmo examina cada encomenda uma vez, realizaremos oito exames.

Pergunta orientadora

O que deve permanecer fixo e o que pode crescer quando aumentamos n?

Operação básica versus tempo total

Função de contagem

$C(n)$ indica o **número de execuções de uma operação selecionada**.

- A contagem pode representar comparações, leituras ou gravações.
- O tempo total ainda inclui controle de laços, acesso a dados e outras instruções.
- Uma operação escolhida **não deve ser presumida constante** se processa uma estrutura de tamanho variável.

Critério de rigor

Em exemplos de enumeração, contar candidatos produzidos não é automaticamente igual ao **tempo total** para materializar ou avaliar todos os candidatos.

Procedimento de derivação

1. **Descrever** a tarefa e definir a entrada.
2. **Escolher** a operação elementar a ser contada.
3. **Simular** para n pequeno, registrando cada execução.
4. **Expressar** o padrão como soma, produto ou recorrência.
5. **Generalizar** a contagem para $C(n)$.
6. **Provar** os limites assintóticos com as definições formais.

Não se deve atribuir uma classe apenas pela aparência do código.

Oito tarefas de uma mesma organização

Caso	Tarefa concreta	Contagem esperada
A	Consultar um pedido por posição conhecida	1
B	Conferir todas as faturas vencidas	n
C	Comparar pares de cadastros para duplicidades	$n(n-1)/2$
D	Avaliar triplas fornecedor–transportadora–janela	n^3
E	Localizar código em registros ordenados	$\lfloor \log_2 n \rfloor + 1$ (pior caso)
F	Ordenar pedidos por intercalação	$n \lceil \log_2 n \rceil$
G	Enumerar pacotes opcionais de serviços	2^n
H	Enumerar a ordem de entregas	$n!$

Cada contagem corresponde à operação indicada; os custos completos podem ter termos adicionais.

Caso A | consulta imediata

Situação operacional

Um atendente recebe o índice de uma encomenda dentro de um vetor e consulta diretamente seu identificador.

- **Entrada:** vetor não vazio e índice válido
- **Tamanho:** n = quantidade de encomendas
- **Operação contabilizada:** uma leitura da posição indicada

Questão de análise

Quantas execuções da operação contabilizada são necessárias quando a entrada cresce?

Pseudocódigo

```
consulta_por_indice(pedidos, pos):
    codigo = pedidos[pos]  # 1 leitura
    retornar codigo
```

Leitura da implementação

O endereço da posição é conhecido: não é necessário percorrer os outros registros.

n	1	5	20	1.000
Leituras da posição	1	1	1	1

1. O algoritmo acessa exatamente uma posição.
2. Não existe repetição vinculada a n.
3. O aumento do vetor não altera a quantidade de leituras analisada.

Função de contagem

$$\boxed{C_A(n) = 1 \quad (n \geq 1)}$$ crescimento constante.

Caso B | conferência de faturas

Situação operacional

O departamento financeiro precisa verificar se cada fatura de uma lista está vencida.

- **Entrada:** vetor de n faturas
- **Tamanho:** n = número de faturas
- **Operação contabilizada:** um teste de vencimento por fatura

Questão de análise

Quantas execuções da operação contabilizada são necessárias quando a entrada cresce?

Pseudocódigo

```
contar_vencidas(faturas):
    total = 0
    para i = 0 ate n-1:
        se vencida(faturas[i]):  # 1 teste
            total = total + 1
    retornar total
```

Leitura da implementação

A condição é verificada uma vez para cada registro, mesmo quando a fatura está em dia.

Caso B | Execução para $n = 4$

Fatura 1	Fatura 2	Fatura 3	Fatura 4
Teste 1	Teste 2	Teste 3	Teste 4

1. O índice inicia em 0.
2. O teste ocorre nas posições $0, 1, 2, 3$.
3. Logo, para $n = 4$, são realizados **4 testes**.
4. Para qualquer n, os índices são $0, 1, \ldots, n - 1$.

Função de contagem

$\boxed{C_B(n) = n}$ crescimento linear.

Caso C | identificação de cadastros duplicados

Situação operacional

Um sistema compara cada par de clientes para detectar possíveis cadastros duplicados.

- **Entrada:** lista de n cadastros
- **Tamanho:** n = número de clientes cadastrados
- **Operação contabilizada:** uma comparação por par distinto, sem repetir o par

Questão de análise

Quantas execuções da operação contabilizada são necessárias quando a entrada cresce?

Pseudocódigo

```
verificar_pares(clientes):
    para i = 0 ate n-1:
        para j = i+1 ate n-1:
            comparar(clientes[i], clientes[j])
```

Leitura da implementação

Os pares (i, j) satisfazem $i < j$: não se compara um cliente consigo mesmo, nem se repete um par invertido.

Caso C | Listagem completa para $n = 4$

Índice externo	Pares comparados	Quantidade
$i = 0$	$(0,1), (0,2), (0,3)$	3
$i = 1$	$(1,2), (1,3)$	2
$i = 2$	$(2,3)$	1
$i = 3$	nenhum	0
	Total	$3 + 2 + 1 + 0 = 6$

Generalização

Com n clientes: $(n-1) + (n-2) + \cdots + 1 + 0$ comparações.

$$\begin{aligned} C_C(n) &= 0 + 1 + 2 + \cdots + (n-1) \\ &= \frac{(n-1)n}{2} \\ &= \frac{n^2 - n}{2}. \end{aligned}$$

Verificação para $n = 4$

$C_C(4) = \frac{4 \cdot 3}{2} = 6$, de acordo com a listagem anterior.

Função exata

$$\boxed{C_C(n) = \frac{n(n-1)}{2}}$$

termo dominante: n^2.

Caso D | compatibilidade operacional

Situação operacional

A empresa examina combinações de um fornecedor, uma transportadora e uma janela de coleta.

- **Entrada:** três conjuntos independentes com n opções cada
- **Tamanho:** n = opções em cada categoria
- **Operação contabilizada:** uma verificação por tripla

Questão de análise

Quantas execuções da operação contabilizada são necessárias quando a entrada cresce?

Pseudocódigo

```
verificar_triplas(F, T, J):
    para cada fornecedor f em F:
        para cada transportadora t em T:
            para cada janela j em J:
                avaliar_compatibilidade(f, t, j)
```

Leitura da implementação

A cada opção de fornecedor correspondem n transportadoras; para cada par há n janelas.

Caso D | Multiplicando as escolhas

1. Escolher o fornecedor: n possibilidades.
2. Para cada fornecedor, escolher transportadora: n possibilidades.
3. Para cada par, escolher janela: n possibilidades.

$$\underbrace{n}_{\text{fornecedores}} \times \underbrace{n}_{\text{transportadoras}} \times \underbrace{n}_{\text{janelas}} = n^3$$

n	2	3	4
$C_D(n)$	8	27	64

Função exata

$\boxed{C_D(n) = n^3}$ crescimento cúbico.

Caso E | localização de um pedido

Situação operacional

Os identificadores das encomendas estão ordenados, permitindo busca binária.

- **Entrada:** vetor ordenado de n identificadores
- **Tamanho:** n = número de registros ordenados
- **Operação contabilizada:** comparações com o elemento central no pior caso

Questão de análise

Quantas execuções da operação contabilizada são necessárias quando a entrada cresce?

Caso E | Algoritmo e operação básica

Pseudocódigo

```
busca_binaria(registros, alvo):
    inicio = 0; fim = n-1
    enquanto inicio <= fim:
        meio = (inicio+fim) // 2
        comparar(alvo, registros[meio])
        se iguais: retornar meio
        se alvo < registros[meio]: fim = meio-1
        senao: inicio = meio+1
    retornar NAO_ENCONTRADO
```

Leitura da implementação

Uma comparação elimina aproximadamente metade da faixa restante; a contagem depende da posição procurada.

Caso E | Simulação: oito registros

Chaves ordenadas

$[10, 20, 30, 40, 50, 60, 70, 80]$; procurar **80**.

Etapa	Intervalo válido	Meio	Decisão
1	posições 0 a 7	40	continuar à direita
2	posições 4 a 7	60	continuar à direita
3	posições 6 a 7	70	continuar à direita
4	posição 7	80	encontrado

Quatro comparações: $8 \to 4 \to 2 \to 1$ candidatos até localizar o item.

Caso E | Da divisão sucessiva ao logaritmo

1. Após k reduções, o intervalo tem tamanho da ordem de $n/2^k$.
2. Uma única posição permanece quando $n/2^k \leq 1$.
3. Logo, $2^k \geq n$, ou $k \geq \log_2 n$.
4. Contando a última comparação e as divisões inteiras, o máximo é:

Número de comparações no pior caso

$$\boxed{C_E(n) = \lfloor \log_2 n \rfloor + 1 \quad (n \geq 1)}$$

Escopo da função

Essa é a **contagem de comparações no pior caso**; buscas que terminam antes executam menos comparações.

Caso F | ordenação de pedidos

Situação operacional

O sistema precisa organizar pedidos por data usando *Merge Sort* iterativo, por intercalações sucessivas.

- **Entrada:** vetor com n pedidos
- **Tamanho:** n = quantidade de pedidos
- **Operação contabilizada:** gravações de elementos no vetor de saída

Questão de análise

Quantas execuções da operação contabilizada são necessárias quando a entrada cresce?

Pseudocódigo

```
ordenar_pedidos(pedidos):
    largura = 1
    enquanto largura < n:
        para cada bloco de tamanho ate 2*largura:
            intercalar_ou_copiar_no_buffer(bloco)
        trocar(origem, buffer)
        largura = 2*largura
```

Leitura da implementação

Na variante definida, até o bloco remanescente é copiado: cada rodada grava exatamente n elementos no buffer.

Caso F | Três rodadas para oito pedidos

Rodada	Largura	Blocos processados	Gravações
1	1	pares de 1 item	8
2	2	pares de 2 itens	8
3	4	pares de 4 itens	8
		Total	24

Por que três rodadas?

$1 \to 2 \to 4 \to 8$ elementos por bloco: foram necessários três dobramentos para atingir $n = 8$.

Caso F | Derivação de $n \log n$

1. Em cada rodada, são gravados n **elementos**.
2. O tamanho dos blocos dobra: $1, 2, 4, \ldots$.
3. Para atingir pelo menos n elementos, são necessárias $\lceil \log_2 n \rceil$ rodadas.
4. Multiplicam-se gravações por rodada e quantidade de rodadas.

Contagem exata na variante definida

$$\boxed{C_F(n) = n\lceil \log_2 n \rceil \quad (n \geq 1)}$$

A contagem é de gravações; comparações, alocações e outras instruções não estão incluídas.

Caso G | combinações de serviços

Situação operacional

Um cliente escolhe livremente quais dos n serviços opcionais incluir no contrato de entrega.

- **Entrada:** lista de n serviços independentes
- **Tamanho:** n = quantidade de serviços opcionais
- **Operação contabilizada:** um registro para cada subconjunto considerado

Questão de análise

Quantas execuções da operação contabilizada são necessárias quando a entrada cresce?

Pseudocódigo

```
listar_pacotes(servicos):
    para cada escolha de n bits:
        registrar_subconjunto_representado(bits)
```

Leitura da implementação

Para cada serviço existem duas alternativas: incluído (1) ou excluído (0). A escolha vazia também é considerada.

Caso G | Exemplo com três serviços

0 0 0	0 0 1	0 1 0	0 1 1	1 0 0	1 0 1	1 1 0	1 1 1

Contagem das escolhas

Três opções independentes, duas decisões por opção: $2 \times 2 \times 2 = 8$ subconjuntos.

n	0	1	2	3	4
Subconjuntos	1	2	4	8	16

Caso G | Construindo o crescimento exponencial

1. Para um serviço existem 2 escolhas possíveis.
2. Para dois serviços existem $2 \cdot 2 = 4$ escolhas.
3. Cada novo serviço duplica o total de subconjuntos.
4. Com n serviços independentes, existem n fatores iguais a 2.

Quantidade de candidatos produzidos

$$\boxed{C_G(n) = 2^n}$$ crescimento exponencial.

Distinção necessária

Produzir ou avaliar *integralmente* um subconjunto pode demandar trabalho adicional; a função conta **quantos candidatos existem**.

Caso H | roteirização de entregas

Situação operacional

O responsável pela logística deseja examinar todas as ordens possíveis de visita a n paradas distintas.

- **Entrada:** conjunto de n paradas; depósito inicial fixo
- **Tamanho:** n = quantidade de paradas
- **Operação contabilizada:** um registro por permutação das paradas

Questão de análise

Quantas execuções da operação contabilizada são necessárias quando a entrada cresce?

Pseudocódigo

```
listar_roteiros(paradas):
    para cada permutacao das n paradas:
        registrar_uma_ordem_de_visita()
```

Leitura da implementação

Cada roteiro corresponde a uma ordem distinta de visita das n paradas; manter o depósito fixo evita contar rotações da mesma lista.

Caso H | Seis roteiros para três paradas

A–B–C	A–C–B	B–A–C
B–C–A	C–A–B	C–B–A

1. Primeira parada: 3 possibilidades.
2. Segunda parada: 2 possibilidades restantes.
3. Terceira parada: 1 possibilidade restante.

$$3 \times 2 \times 1 = 6 = 3!$$

Caso H | Produto decrescente e fatorial

- Para n paradas, a primeira escolha oferece n opções.
- A segunda oferece $n-1$ opções.
- O número de opções diminui até restar apenas uma.

$$C_H(n) = n(n-1)(n-2)\cdots 2 \cdot 1 = n!$$

n	2	3	4	5
$C_H(n)$	2	6	24	120

Escopo

A quantidade de roteiros é $n!$; calcular distância total de cada roteiro acrescenta custo, em geral proporcional a n.

As oito funções obtidas

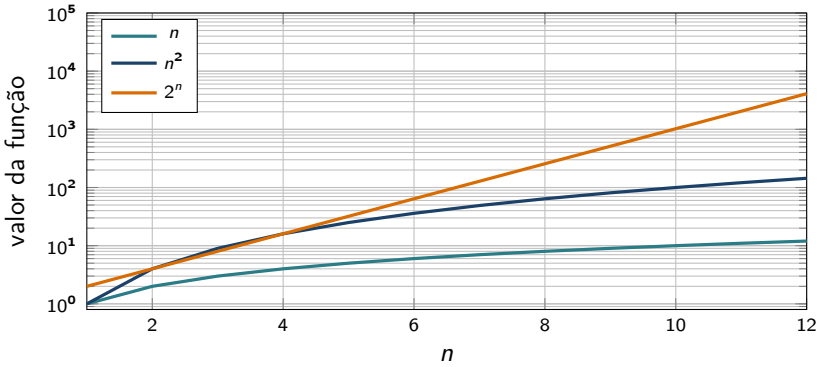
Caso	Contagem $C(n)$	Interpretação
A	1	consulta por índice
B	n	verificação de todas as faturas
E	$\lfloor \log_2 n \rfloor + 1$	comparações da busca binária no pior caso
F	$n\lceil \log_2 n \rceil$	gravações da intercalação iterativa
C	$n(n-1)/2$	pares não ordenados
D	n^3	triplas independentes
G	2^n	pacotes opcionais
H	$n!$	ordens de visita

Funções: valores concretos e ordem de grandeza

$C(n)$	$n = 2$	$n = 4$	$n = 8$	$n = 16$
1	1	1	1	1
n	2	4	8	16
$\lfloor \log_2 n \rfloor + 1$	2	3	4	5
$n\lceil \log_2 n \rceil$	2	8	24	64
$n(n-1)/2$	1	6	28	120
n^3	8	64	512	4096
2^n	4	16	256	65536
$n!$	2	24	40320	$\approx 2{,}09 \cdot 10^{13}$

Comparações numéricas ilustram padrões, mas não constituem demonstração assintótica.

Visualização: crescimento polinomial e exponencial



Eixo vertical logarítmico para permitir visualizar valores de diferentes escalas.

Crescimento não é igual a desempenho medido

O que foi demonstrado até aqui

As expressões descrevem **a quantidade de uma operação escolhida**, sob hipóteses explícitas.

- Duas implementações com a mesma classe podem ter tempos distintos.
- A análise assintótica descreve o comportamento para entradas suficientemente grandes.
- Em comparação real de desempenho, também importam constantes, memória, representação dos dados e hardware.

Próxima etapa

Passaremos de igualdades exatas ($C(n) = \cdots$) para **relações de limites** (O, Ω, Θ e o).

Por que utilizar notações assintóticas?

- Uma função exata pode conter diversos termos e constantes.
- Frequentemente, interessa caracterizar **o crescimento para grandes entradas**.
- As notações estabelecem relações matemáticas entre funções, não valores em segundos.

Exemplo motivador

$T(n) = 5n^2 + 3n + 7$ tem crescimento quadrático, mas é necessário justificar formalmente o limite.

Atenção

A notação O **não significa, por definição, pior caso**; pior caso identifica qual função de custo estamos analisando.

Definição formal: Big O

Limite superior assintótico

$f(n) \in O(g(n))$ se existem $c > 0$ e n_0 tais que

$$0 \leq f(n) \leq c\, g(n) \quad \text{para todo } n \geq n_0.$$

1. Escolha uma função de comparação $g(n)$.
2. Procure uma constante positiva c.
3. Determine a partir de qual n_0 a desigualdade vale.

Interpretação

A partir de n_0, a função $c\, g(n)$ fica acima de $f(n)$.

Definição formal: Ômega

Limite inferior assintótico

$f(n) \in \Omega(g(n))$ se existem $c > 0$ e n_0 tais que

$$0 \leq c\, g(n) \leq f(n) \quad \text{para todo } n \geq n_0.$$

1. Escolha $g(n)$.
2. Encontre um fator positivo c que forme uma cota inferior.
3. Defina um limiar n_0 para o qual a cota permanece válida.

Interpretação

A partir de n_0, a função $c\, g(n)$ fica abaixo de $f(n)$.

Definição formal: Theta

Crescimento assintoticamente justo

$f(n) \in \Theta(g(n))$ se existem $c_1, c_2 > 0$ e n_0 tais que

$$0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n), \quad n \geq n_0.$$

$$\boxed{\Theta(g) = O(g) \cap \Omega(g)}$$

Interpretação

A função está limitada **superior e inferiormente** por múltiplos positivos de $g(n)$.

Definição formal: little-o

Limite superior estrito

$f(n) \in o(g(n))$ se, para **todo** $c > 0$, existe n_0 tal que

$$0 \leq f(n) < c\, g(n) \quad \text{para todo } n \geq n_0.$$

- Não basta encontrar *um único* valor de c, como em O.
- Quando $g(n) > 0$ eventualmente, é suficiente provar:

$$\lim_{n \to \infty} \frac{f(n)}{g(n)} = 0.$$

Distinção

$n^2 \in O(n^2)$, mas $n^2 \notin o(n^2)$.

Procedimento para demonstrar Θ

1. Escreva a expressão exata de $C(n)$.
2. Proponha uma função $g(n)$ com o termo dominante.
3. Mostre $C(n) \leq c_2 g(n)$ para $n \geq n_0$ (cota superior).
4. Mostre $c_1 g(n) \leq C(n)$ para $n \geq n_0$ (cota inferior).
5. Conclua $C(n) \in \Theta(g(n))$.

Depois da prova

Para uma relação de crescimento **estritamente menor**, compare a razão $C(n)/h(n)$ com zero.

Exemplo preliminar: um polinômio completo

Dado $T(n) = 5n^2 + 3n + 7$, para $n \geq 1$:

$$\begin{aligned} T(n) &= 5n^2 + 3n + 7 \\ &\leq 5n^2 + 3n^2 + 7n^2 = 15n^2, \\ T(n) &\geq 5n^2. \end{aligned}$$

Escolha de constantes

$c_1 = 5$, $c_2 = 15$, $n_0 = 1$.

Conclusão

$\boxed{T(n) \in \Theta(n^2)}$; em particular, $T(n) \in O(n^2)$ e $T(n) \in \Omega(n^2)$.

Ponto de partida

$C_A(n) = 1$ para $n \geq 1$.

1. **Big O:** $1 \leq 1 \cdot 1$; use $c_2 = 1$, $n_0 = 1$.
2. **Ômega:** $1 \cdot 1 \leq 1$; use $c_1 = 1$, $n_0 = 1$.
3. Como as duas desigualdades valem, $C_A(n) \in \Theta(1)$.

Resultado

$$\boxed{C_A(n) \in O(1) \cap \Omega(1) = \Theta(1)}$$

Comparação com uma função linear

Tomemos $g(n) = n$. Então

$$\frac{C_A(n)}{n} = \frac{1}{n} \xrightarrow[n\to\infty]{} 0.$$

1. O numerador permanece fixo em 1.
2. O denominador aumenta indefinidamente.
3. A razão tende a zero.

Conclusão

$\boxed{1 \in o(n)}$ crescimento constante é estritamente inferior ao linear.

Ponto de partida

$C_B(n) = n$.

1. $n \leq 1 \cdot n$; logo $C_B(n) \in O(n)$, com $c_2 = 1$.
2. $1 \cdot n \leq n$; logo $C_B(n) \in \Omega(n)$, com $c_1 = 1$.
3. Ambas valem para todo $n \geq 1$: $n_0 = 1$.

Resultado

$\boxed{C_B(n) \in \Theta(n)}$

Prova B | Limite estrito em relação a n^2

$$\begin{aligned}\frac{C_B(n)}{n^2} &= \frac{n}{n^2} \\ &= \frac{1}{n} \longrightarrow 0.\end{aligned}$$

Conclusão

$\boxed{n \in o(n^2)}$.

Importante: $n \in O(n^2)$ também é verdadeiro, mas a classe $\Theta(n)$ caracteriza melhor seu crescimento.

Expressão exata

$$C_C(n) = \frac{n(n-1)}{2} = \frac{n^2 - n}{2}.$$

Para $n \geq 2$, como $n - 1 \leq n$:

$$C_C(n) = \frac{n(n-1)}{2} \leq \frac{n^2}{2}.$$

Big O

Escolha $g(n) = n^2$, $c_2 = \frac{1}{2}$, $n_0 = 2$. Logo $\boxed{C_C(n) \in O(n^2)}$.

Para $n \geq 2$, temos $n - 1 \geq n/2$. Portanto:

$$C_C(n) = \frac{n(n-1)}{2} \geq \frac{n(n/2)}{2} = \frac{n^2}{4}.$$

Combinando as duas cotas:

$$\frac{1}{4}n^2 \leq C_C(n) \leq \frac{1}{2}n^2 \quad (n \geq 2).$$

Ômega e Theta

$c_1 = \frac{1}{4}$, $c_2 = \frac{1}{2}$, $n_0 = 2$; assim $\boxed{C_C(n) \in \Theta(n^2)}$.

$$\begin{aligned}\frac{C_C(n)}{n^3} &= \frac{n(n-1)}{2n^3} \\ &= \frac{n-1}{2n^2} = \frac{1}{2n} - \frac{1}{2n^2} \longrightarrow 0.\end{aligned}$$

Conclusão

$\boxed{C_C(n) \in o(n^3)}$.

Função exata

$C_D(n) = n^3$, para $n \geq 1$.

1. $C_D(n) \leq 1 \cdot n^3$ prova $O(n^3)$.
2. $1 \cdot n^3 \leq C_D(n)$ prova $\Omega(n^3)$.
3. Tomando $c_1 = c_2 = 1$ e $n_0 = 1$, concluímos:

Resultado

$\boxed{C_D(n) \in \Theta(n^3)}$

$$\frac{n^3}{n^4} = \frac{1}{n} \xrightarrow[n\to\infty]{} 0.$$

Conclusão

$\boxed{n^3 \in o(n^4)}$.

Interpretação operacional

Triplicar o número de opções **em cada uma das três categorias** multiplica o total de triplas por $3^3 = 27$.

Contagem no pior caso

$C_E(n) = \lfloor \log_2 n \rfloor + 1$.

Pela propriedade do piso, $\lfloor x \rfloor > x - 1$. Logo:

$$C_E(n) = \lfloor \log_2 n \rfloor + 1 > \log_2 n.$$

Ômega

Para $n \geq 2$, $C_E(n) \geq 1 \cdot \log_2 n$: $\boxed{C_E(n) \in \Omega(\log n)}$.

Pela propriedade do piso:

$$C_E(n) \leq \log_2 n + 1.$$

Para $n \geq 2$, $\log_2 n \geq 1$; então:

$$\log_2 n + 1 \leq 2 \log_2 n.$$

Duas cotas explícitas

$\log_2 n \leq C_E(n) \leq 2 \log_2 n$ para $n \geq 2$. $c_1 = 1$, $c_2 = 2$, $n_0 = 2$, logo $\boxed{C_E(n) \in \Theta(\log n)}$.

Como $C_E(n) \leq 2\log_2 n$ para $n \geq 2$:

$$0 \leq \frac{C_E(n)}{n} \leq 2\frac{\log_2 n}{n} \xrightarrow[n\to\infty]{} 0.$$

Conclusão

$\boxed{C_E(n) \in o(n)}$.

O limite $\log n/n \to 0$ *pode ser demonstrado por propriedades elementares de crescimento ou cálculo.*

Ponto de partida

$C_F(n) = n\lceil \log_2 n \rceil$.

Para $n \geq 2$:

$$\log_2 n \leq \lceil \log_2 n \rceil \leq \log_2 n + 1 \leq 2 \log_2 n.$$

Multiplique todos os termos por $n > 0$:

$$n \log_2 n \leq C_F(n) \leq 2n \log_2 n.$$

Conclusão

$c_1 = 1,\ c_2 = 2,\ n_0 = 2;\ \boxed{C_F(n) \in \Theta(n \log n)}$.

$$0 \leq \frac{C_F(n)}{n^2} \leq \frac{2n \log_2 n}{n^2} = 2\frac{\log_2 n}{n} \longrightarrow 0.$$

Conclusão

$\boxed{C_F(n) \in o(n^2)}$.

Significado prático

A ordenação por intercalação, nessa métrica, cresce estritamente mais lentamente que uma comparação entre todos os pares.

Ponto de partida

$C_G(n) = 2^n$.

1. $2^n \leq 1 \cdot 2^n$ fornece a cota superior.
2. $1 \cdot 2^n \leq 2^n$ fornece a cota inferior.
3. $c_1 = c_2 = 1$ e $n_0 = 0$ justificam ambas as cotas.

Conclusão

$\boxed{C_G(n) \in \Theta(2^n)}$.

$$\frac{2^n}{3^n} = \left(\frac{2}{3}\right)^n \xrightarrow[n\to\infty]{} 0.$$

Conclusão

$\boxed{2^n \in o(3^n)}$.

Observação

A função ainda é exponencial; little-o compara **duas funções específicas**, não transforma uma função exponencial em polinomial.

Ponto de partida

$C_H(n) = n!$ para $n \geq 1$.

1. $n! \leq 1 \cdot n!$ prova $O(n!)$.
2. $1 \cdot n! \leq n!$ prova $\Omega(n!)$.
3. Com $c_1 = c_2 = 1$, $n_0 = 1$:

Conclusão

$\boxed{C_H(n) \in \Theta(n!)}$.

Prova H | Por que $n!$ é estritamente menor que n^n?

Para $n \geq 2$:

$$\frac{n!}{n^n} = \frac{1}{n} \cdot \frac{2}{n} \cdots \frac{n}{n}.$$

Pelo menos $\lfloor n/2 \rfloor$ fatores são $\leq 1/2$; os demais são ≤ 1.

$$0 \leq \frac{n!}{n^n} \leq \left(\frac{1}{2}\right)^{\lfloor n/2 \rfloor} \longrightarrow 0.$$

Conclusão

$\boxed{n! \in o(n^n)}$.

Síntese: as quatro notações nos oito casos

Caso	$C(n)$	Crescimento justo	Exemplo de limite estrito
A	1	$\Theta(1)$	$o(n)$
B	n	$\Theta(n)$	$o(n^2)$
C	$n(n-1)/2$	$\Theta(n^2)$	$o(n^3)$
D	n^3	$\Theta(n^3)$	$o(n^4)$
E	$\lfloor \log_2 n \rfloor + 1$	$\Theta(\log n)$	$o(n)$
F	$n\lceil \log_2 n \rceil$	$\Theta(n \log n)$	$o(n^2)$
G	2^n	$\Theta(2^n)$	$o(3^n)$
H	$n!$	$\Theta(n!)$	$o(n^n)$

Em cada linha, a prova de Θ implica uma prova de O e de Ω para a função indicada.

Por que O não fornece necessariamente a melhor cota?

Caso linear

Uma varredura possui $C(n) = n$.

Para $n \geq 1$:

$$n \leq n^2 \quad \Rightarrow \quad C(n) \in O(n^2).$$

Entretanto, n^2 não expressa seu crescimento justo:

$$\frac{n}{n^2} = \frac{1}{n} \to 0.$$

Classificação mais informativa

$C(n) \in \Theta(n)$, mesmo que $C(n) \in O(n^2)$ também seja verdadeiro.

Cuidado: pior caso e notação assintótica

Busca binária

$C_{\text{pior}}(n) = \lfloor \log_2 n \rfloor + 1 \in \Theta(\log n)$.

Busca com sucesso imediato

Se o elemento procurado coincide com o primeiro elemento central, a execução usa apenas uma comparação: $C_{\text{melhor}}(n) = 1 \in \Theta(1)$.

Distinção

Melhor/pior caso especifica o cenário da entrada; O, Ω e Θ descrevem limites da *função* definida para esse cenário.

Exercício 1 | Contagem de inspeções

Problema

Um sistema verifica todas as encomendas e depois imprime um relatório de resumo.

```
para i = 0 ate n-1:
    conferir_pedido(i)
gerar_relatorio_resumido()
```

1. Quantas chamadas a `conferir_pedido` são realizadas?
2. Quantas chamadas a `gerar_relatorio_resumido`?
3. Se ambas tiverem custo constante, qual a classe Θ do tempo total?

Exercício 1 | Resolução

- O laço realiza n verificações.
- O relatório de resumo é gerado uma vez.
- Uma métrica de operações unitárias simplificada é $T(n) = n + 1$.

Para $n \geq 1$:

$$n \leq n + 1 \leq 2n.$$

Conclusão

Com $c_1 = 1$, $c_2 = 2$, $n_0 = 1$: $\boxed{T(n) \in \Theta(n)}$.

Exercício 2 | Comparações por pares

Problema

Uma escola cruza pares de inscrições para identificar possíveis homônimos.

```
para i = 0 ate n-1:
    para j = i+1 ate n-1:
        comparar(i,j)
```

1. Liste os pares analisados para $n = 5$.
2. Obtenha a soma de comparações.
3. Prove uma cota inferior e superior quadrática.

Exercício 2 | Resolução

1. Contagens por índice externo: $4, 3, 2, 1, 0$.
2. Soma: $4 + 3 + 2 + 1 + 0 = 10$.
3. Generalização: $C(n) = n(n-1)/2$.
4. Para $n \geq 2$:

$$\frac{1}{4}n^2 \leq C(n) \leq \frac{1}{2}n^2.$$

Conclusão

$\boxed{C(n) \in \Theta(n^2)}$.

Exercício 3 | Busca em uma lista ordenada

Dados

Códigos ordenados: $[5, 9, 14, 18, 27, 35, 42, 60, 75]$.

1. Execute a busca binária pelo código 75.
2. Registre os intervalos e elementos centrais consultados.
3. Explique por que a quantidade máxima de comparações cresce como $\Theta(\log n)$.

Condição

Utilize índice inicial 0, índice final 8 e meio calculado por divisão inteira.

Exercício 3 | Resolução

Etapa	Intervalo	Meio	Valor
1	$[0, 8]$	4	27
2	$[5, 8]$	6	42
3	$[7, 8]$	7	60
4	$[8, 8]$	8	75

Foram realizadas quatro comparações. Em geral, o tamanho da faixa remanescente é aproximadamente dividido por dois a cada etapa.

$$\boxed{C_{\text{pior}}(n) = \lfloor \log_2 n \rfloor + 1 \in \Theta(\log n)}$$

Exercício 4 | Ordenação e crescimento

Tarefa

Na ordenação iterativa por intercalação, um algoritmo grava os n registros a cada rodada e dobra o tamanho dos blocos.

1. Quantas rodadas existem para $n = 16$?
2. Quantas gravações em cada rodada?
3. Qual é a contagem exata, nessa implementação?
4. Justifique as cotas que permitem concluir $\Theta(n \log n)$.

Exercício 4 | Resolução

1. Blocos com larguras $1, 2, 4, 8$; são quatro rodadas.
2. Em cada rodada: 16 gravações.
3. Contagem: $C(16) = 16 \cdot 4 = 64$.
4. No caso geral: $C(n) = n\lceil \log_2 n \rceil$.

Para $n \geq 2$:

$$n \log_2 n \leq C(n) \leq 2n \log_2 n.$$

Resultado

$\boxed{C(n) \in \Theta(n \log n)}$.

Produção individual

Escolha **um** processo computacional entre:

1. Conferir todos os registros de uma tabela.
2. Comparar cada par distinto de registros.
3. Encontrar um código em lista ordenada.
4. Enumerar todas as combinações de opções independentes.

Entregue: **definição de** n, **pseudocódigo**, **operação básica**, **simulação para entrada pequena**, $C(n)$ e **prova de** Θ.

1. Definir a entrada e explicitar as hipóteses do problema.
2. Selecionar a operação básica e contabilizá-la.
3. Derivar a função $C(n)$ sem depender de exemplos isolados.
4. Demonstrar O e Ω com constantes e limiar n_0.
5. Concluir Θ quando as duas cotas coincidirem em ordem.
6. Empregar o quando uma razão de funções tende a zero.

Objetivo final

A classificação assintótica deve resultar de uma demonstração.